future internet

MDPI

Article

C-GUARD: Context-Adaptive Conformal Gating for Improving Robustness Against Evasive Windows PE Malware

Muhammad Imran [1,*], Malik Al-Essa [2], Felice Franchini [1] and Giuseppe Pirlo [1,*]

[1] Department of Computer Science, Università Degli Studi di Bari Aldo Moro, via Orabona, 4, 70125 Bari, Italy; f.franchini9@phd.uniba.it
[2] Department of Computer Science, King Abdullah II School for Information Technology, University of Jordan, Amman 11942, Jordan; m.alessa@ju.edu.jo
* Correspondence: muhammad.imran@uniba.it (M.I.); giuseppe.pirlo@uniba.it (G.P.)

Abstract

Machine Learning (ML)-based malware detectors perform exceptionally well on standard benchmarks, but their robustness to evasive malware remains fragile. On EMBER2024, strong tree-based models achieve high accuracy on clean test sets yet miss a large number of evasive Windows PE samples. Through SHAP-based explainability, matched error comparisons, feature-family analysis, and margin diagnostics on the validation split, we identify consistent attribution and decision-margin differences between baseline true negatives and false negatives. We further show that false negatives occur at different decision depths, including both near-boundary cases and samples misclassified with high benign confidence. We propose C-GUARD, a context-adaptive conformal gated detector that preserves the optimized high-performing baseline while selectively invoking an auxiliary rescue detector on a targeted subset of baseline negative predictions. C-GUARD combines out-of-fold rescue learning to distinguish baseline false negatives from true negatives with adaptive gating under an explicit false-positive budget. A one-sided intervention rule preserves all baseline malware decisions. On the EMBER2024 standard test set, C-GUARD recovers 178 additional malware samples at the cost of 82 additional false positives. On the evasive challenge set, it recovers 17 additional evasive malware samples.

Keywords: explainable artificial intelligence; adversarial machine learning; evasive malware detection; Windows PE malware; model robustness; conformal prediction; AI security; machine learning security

Check for updates

Academic Editors: Wei Zong and Yang-Wai Chow

Received: 21 June 2026
Revised: 9 August 2026
Accepted: 11 August 2026
Published: 13 August 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

ML and DL have become dominant approaches for automated malware detection, driven by their ability to learn complex decision boundaries from large-scale data and to outperform signature-based systems on previously unseen samples [1,2]. In particular, static Windows Portable Executable (PE) classifiers trained on engineered feature sets have achieved near-saturated performance on widely used benchmarks, suggesting that high-accuracy detection at scale is feasible. Yet, in security practice, high benchmark accuracy does not guarantee robust protection: adversaries actively modify malware to evade detection while preserving its malicious behavior, and such evasive inputs often differ systematically from the distributions represented in standard test splits.

Recent large-scale evaluations highlight this gap. On the EMBER2024 [3] benchmark, state-of-the-art tree-based detectors obtain excellent results on conventional evaluation sets but exhibit a surprisingly high number of false negatives on challenge-evasive PE malware. These failures are not merely rare corner cases. Instead, they reflect a broader and more operationally relevant weakness: many high-performing models remain vulnerable under manipulations that are inexpensive for attackers and difficult to anticipate through standard training alone [4,5]. This vulnerability is especially concerning in real deployments, where systems are typically tuned to extremely low false-positive rates (FPRs) in order to avoid overwhelming analysts. Any robustness improvement that significantly increases false positives may be unacceptable, even if it improves true-positive detection in aggregate. A central obstacle to addressing evasive failures is that modern detectors often behave as black boxes [6]. Without principled diagnostic analysis, it is difficult to determine whether a model relies on robust discriminative evidence or on feature patterns that fail to generalize to evasive samples. Explainability methods can support this analysis by revealing attribution and margin patterns associated with model failures, which should be validated through controlled ablations rather than interpreted as direct causal evidence [7].

In this work, we investigate why strong tree-based Windows PE malware detectors fail on evasive samples and how to mitigate such failures under a constrained false-positive budget. We conduct an explainability-driven diagnostic analysis on validation false negatives and true negatives using SHapley Additive exPlanations (SHAP) [8], feature-family analysis, and margin-based diagnostics to characterize failure-associated attribution and decision margin patterns. Based on this analysis, we develop C-GUARD, a context-adaptive conformal gated framework that selectively re-evaluates baseline negative predictions through an auxiliary rescue detector. The main contributions of this paper are the following:

- We provide an explainability-driven characterization of false negatives with matched true negatives on validation samples, showing consistent differences in feature attribution and decision-margin behavior across several PE feature families.
- We show that baseline false negatives occur at different decision depths, including near-boundary cases and samples misclassified with high benign confidence, highlighting the heterogeneity of baseline failure behavior.
- We introduce C-GUARD, a context-adaptive conformal gated framework that invokes an auxiliary rescue detector on selected baseline negative predictions while preserving all baseline malware decisions and maintaining a low false-positive operating point.
- We empirically evaluate C-GUARD on the EMBER2024 standard test and challenge sets, demonstrating additional malware recovery with only a small increase in false positives.
- We conduct controlled ablations across feature representations, auxiliary architectures, and ensemble strategies, confirming the full-feature EMBER LightGBM auxiliary as the strongest overall rescue configuration.

The rest of this paper is organized as follows. Section 2 reviews the most relevant prior work. Section 3 details the proposed approach. Section 4 presents the datasets, outlines the implementation and experimental setup, and discusses the results and diagnostic ablations. Finally, Section 5 concludes by summarizing the main contributions and highlighting directions for future research.

2. Literature Review

2.1. Explainable AI for PE Malware Detection

Explainable AI (XAI) has been widely adopted to interpret static malware detection models operating on Windows Portable Executable (PE) files. A substantial body of work applies post hoc feature attribution methods, including SHAP [8] and LIME [9], to quantify the contribution of individual PE features to classification decisions. These approaches primarily aim to improve transparency, analyst trust, and post hoc interpretability by visualizing feature importance or interaction effects, without modifying the underlying feature space, decision boundary, or deployment behavior of the detector [10]. A limited subset of studies additionally performs lightweight feature reduction or efficiency-oriented retraining based on importance rankings [11,12] but still treats explainability as an auxiliary interpretive tool rather than as a mechanism for robustness analysis or failure diagnosis [13]. Beyond post hoc explanations, several works investigate intrinsically explainable malware detection architectures for Windows PE files. These include generative models that produce natural-language explanations [14], memory-based detectors offering similarity-driven explanations [15], and graph-based approaches that extract interpretable substructures from executable representations [16]. MDOB employs SHAP to identify redundant or low-impact PE features and retrains ensemble classifiers on reduced feature sets, implicitly assuming that compact representations improve robustness [17]. In parallel, a distinct line of research exploits XAI to guide adversarial and evasion attacks against malware detectors [18]. In feature-space settings, explanation methods such as SHAP [8], LIME [9], Integrated Gradients [19], and LRP [20] are used to identify influential features and construct minimal perturbations for evasion [21,22]. More recent studies extend this concept to the input space by mapping explanation scores to raw PE binaries, identifying decision-critical regions where functionality-preserving manipulations can be applied [23]. In these works, explainability functions explicitly as an attack-guidance mechanism rather than as a diagnostic or defensive tool.

The method proposed in this work adopts XAI as a *diagnostic instrument* to analyze why a high-performing PE malware detector fails under evasive conditions on a public benchmark. Rather than using explanations to guide attacks or perform global feature elimination, we combine SHAP attribution, matched error comparisons, feature-family analysis, and decision-margin diagnostics to identify failure-associated differences between false negatives and true negatives in the validation split. The findings are subsequently examined through controlled representation, feature-removal, architecture, and ensemble ablations.

2.2. Evasive and Adversarial Manipulation of Windows PE Malware

Early problem-space attacks focus on modifying unused or non-critical regions of PE binaries while preserving executability and malicious behavior. [24] introduced Full DOS, Extend DOS, and Content SHIFT attacks, which manipulate PE header regions to evade static detectors under a white-box setting. Subsequent work extends this paradigm by injecting data into slack spaces or appending new sections guided by prototype bytes [25] or by exploiting intra-section code caves to achieve higher evasion rates than simple end-of-file appending [26]. More advanced white-box problem-space attacks optimize localized binary patches. MalPatch [27] formulates evasion as a constrained optimization problem and applies projected gradient descent to generate masked binary patches within fixed PE regions. Other studies confirm that even detectors operating on alternative representations, such as image-based encodings of PE files, remain vulnerable to adversarial manipulations applied through executable-safe binary transformations [28]. Instruction-level transformations and code relocation into newly injected sections further expand the space of functionality-preserving attacks [29].

Parallel to white-box methods, black-box adversarial frameworks have been proposed to generate evasive PE malware without access to model gradients. Reinforcement learning and evolutionary strategies are commonly employed to select sequences of executable-safe actions, such as section injection, import table manipulation, overlay appending, and header modification [30,31]. Multi-armed bandit formulations [32] and GAN-based feature-space attacks [33] further demonstrate that evasion is feasible even under severely restricted feedback. MTMG [34] extends this line of work by targeting multiple learning-based malware detectors simultaneously, highlighting the transferability of evasive behavior across different model architectures.

3. C-GUARD Framework

C-GUARD is a hybrid detection framework composed of three tightly coupled components, as illustrated in Algorithm 1 and Figure 1:

- **Baseline detector (f):** an Optuna-optimized LightGBM model trained on the complete EMBER feature space (2568 features). The baseline model is responsible for all predictions by default and defines the deployment operating point through a fixed threshold τ_b corresponding to FPR $= 0.01$ on the validation benign set.
- **Auxiliary rescue detector (g):** a conditional LightGBM classifier trained to distinguish true negatives from false negatives within the baseline-negative region. Its training data were constructed through three-fold out-of-fold scoring. In each iteration, the baseline was trained on two folds, and eligible baseline-negative samples from the held-out fold were added to the rescue-training set. The rescue detector operates on the complete EMBER feature space.
- **Adaptive conformal gate ($\mathcal{G}$):** a validation-selected, context-conditioned gating mechanism that determines eligible baseline-negative samples for auxiliary intervention. The gate first assigns each baseline-negative sample to a validation-derived context using the rescue-model score. It then compares the sample's baseline-score behavior with that of known validation false negatives in the same context to obtain a class-conditional conformal compatibility value.

C-GUARD enforces an asymmetric intervention policy: the rescue detector is allowed to override baseline benign predictions, but it is never permitted to overturn baseline malware decisions. This design reflects the operational priority of minimizing false positives while recovering evasive malware.

The baseline detector is trained on the complete training distribution to solve the global benign versus malware classification task. In contrast, the rescue detector is trained on a dedicated rescue dataset. Let $\mathcal{D}_{train} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the training split. To construct the rescue dataset, the training split is partitioned into three stratified folds. In each iteration, a fold-specific LightGBM detector using the optimized baseline configuration is trained on two folds and used to score the remaining held-out fold. In this way, each training sample receives one out-of-fold score from a model that was not trained on that sample. Using these out-of-fold scores, the canonical rescue set is defined as follows:

$$\mathcal{D}_{res} = \left\{ (x_i, z_i) \;\middle|\; x_i \in \mathcal{R}_{OOF},\ z_i = \begin{cases} 0, & y_i = 0, \\ 1, & y_i = 1, \end{cases} \right\}, \tag{1}$$

where $\mathcal{R}_{OOF}$ denotes the baseline-negative out-of-fold rescue region, and z_i distinguishes eligible true negatives from eligible false negatives. The rescue dataset contains 4297 samples, comprising 2106 true negatives and 2191 false negatives. The auxiliary rescue detector g is trained using all 2568 features of this dataset and maps each feature vector in $\mathbb{R}^{2568}$ to a rescue score in $[0, 1]$. Notably, although both detectors use the complete EMBER feature space, the auxiliary rescue detector task is to identify false negatives among the samples classified as benign by the baseline detector.

Explainability is employed as a diagnostic mechanism to understand the failure behavior of the baseline detector. TreeSHAP analysis is performed on true-negative and false-negative samples from the validation split, and the resulting evidence is used to perform representation, sufficiency, and feature-removal experiments.

To identify baseline-negative samples for intervention, C-GUARD employs a validation-only adaptive conformal gate. Let $s_b(x)$ denote the raw score produced by the baseline detector, and let τ_b denote the frozen baseline threshold. The corresponding sigmoid-transformed baseline score is defined as $p_b(x) = 1/(1 + \exp(-s_b(x)))$. Only samples satisfying $s_b(x) < \tau_b$ are considered for intervention.

For each baseline-negative sample, the auxiliary rescue detector produces a rescue score $r(x) = g(x)$. This score is used to assign the sample to one of k quantile-based contexts derived from the validation calibration data, where k denotes the number of rescue-score contexts. Within the assigned context, the malware class nonconformity score is defined as $a_1(x) = 1 - p_b(x)$.

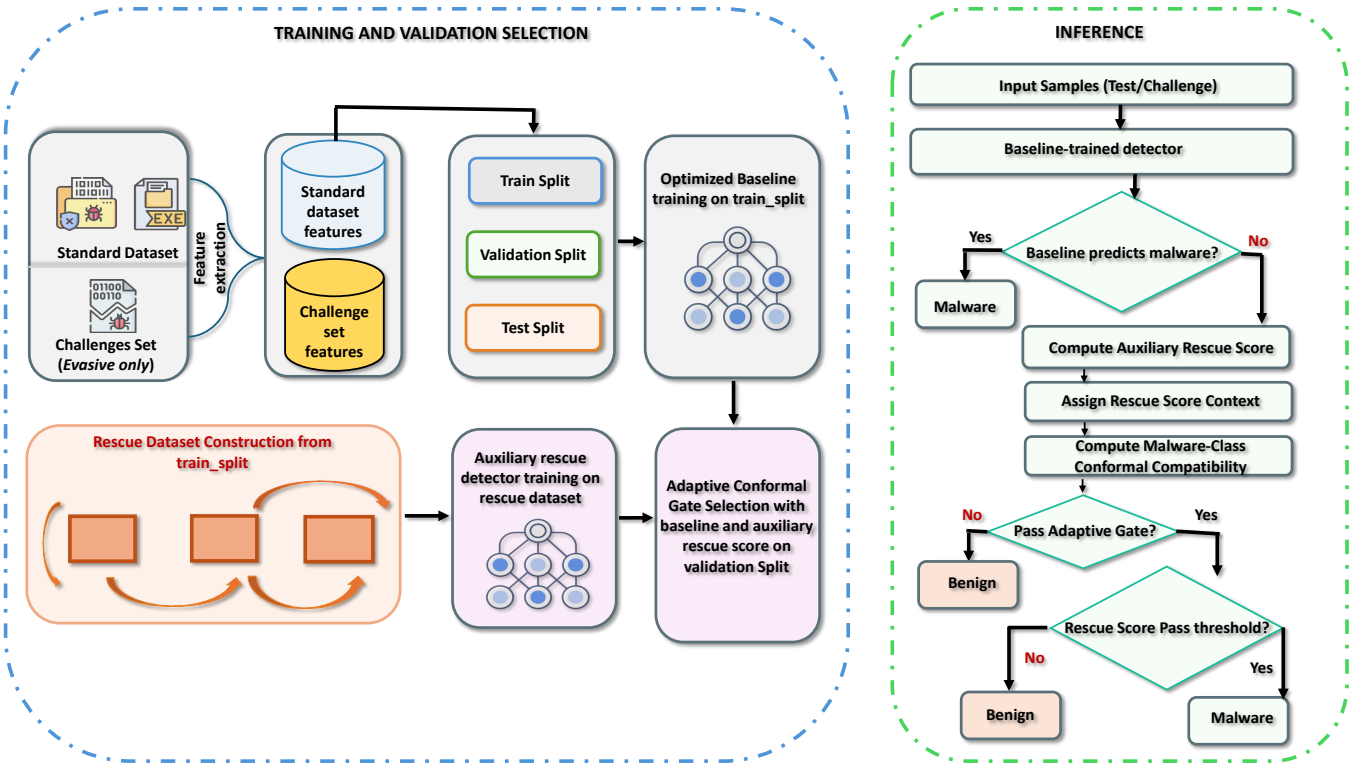


Figure 1. Overview of the proposed method.

Algorithm 1: C-GUARD: Context-Adaptive Conformal Gated Detection

Input : $\mathcal{D}_{train} = \{(x_i, y_i)\}$: training split feature vectors ($x_i \in \mathbb{R}^{2568}$); $\mathcal{D}_{val} = \{(x_i, y_i)\}$: validation split; $\mathcal{D}_{test} = \{(x_i, y_i)\}$: standard test set; $\mathcal{D}_{ch} = \{(x_i, 1)\}$: challenge set (evasive malware only); Θ_f: optimized LightGBM baseline configuration; τ_b: baseline threshold at FPR = 0.01; ϵ_{res}: benign fraction (define each out-of-fold rescue region); $\mathcal{K}$: candidate numbers of rescue-score contexts; B_{FP}: additional false-positive budget; B_{gate}: maximum number of gated validation true negatives; $n_{\min}$: minimum local class-calibration size.

Output: Baseline detector f, auxiliary rescue detector g, conformal calibrator $\mathcal{C}^*$, selected parameters $(k^*, \alpha^*, \tau_r^*)$; and confusion matrices

Function ConformalPValue$(x, r, s_b, \mathcal{C}^*)$:
- $j \leftarrow$ AssignContext$(r, \mathcal{C}^*)$ `// frozen rescue-score context`
- $p_b \leftarrow 1/(1 + \exp(-s_b))$
- $a_1 \leftarrow 1 - p_b$ `// malware-class nonconformity`
- $\mathcal{A} \leftarrow \mathcal{A}^*_{1,j}$
- **if** $|\mathcal{A}| < n_{\min}$ **then**
  - $\mathcal{A} \leftarrow \mathcal{A}^*_{1,\text{global}}$ `// global class-conditional fallback`
- $p_1 \leftarrow \dfrac{1 + |\{a \in \mathcal{A} : a \geq a_1\}|}{|\mathcal{A}| + 1}$
- **return** p_1

Function CGUARD_Decision(x):
- $s_b \leftarrow f_{\text{raw}}(x)$
- **if** $s_b \geq \tau_b$ **then**
  - **return** 1
- $r \leftarrow g(x)$
- $p_1 \leftarrow$ ConformalPValue$(x, r, s_b, \mathcal{C}^*)$
- **if** $p_1 > \alpha^*$ ***and*** $r \geq \tau_r^*$ **then**
  - **return** 1 `// baseline-negative sample promoted to malware`
- **return** 0

`// Step 1: Construct the out-of-fold rescue dataset`

Partition $\mathcal{D}_{train}$ into three stratified folds $\{\mathcal{F}_1, \mathcal{F}_2, \mathcal{F}_3\}$

$\mathcal{D}_{res} \leftarrow \emptyset$

for $q \leftarrow 1$ **to** 3 **do**
- Train a fold-specific LightGBM detector $f^{(q)}$ using configuration Θ_f on $\mathcal{D}_{train} \setminus \mathcal{F}_q$
- Compute OOF raw scores $s_i^{(q)} \leftarrow f_{\text{raw}}^{(q)}(x_i)$ for all $(x_i, y_i) \in \mathcal{F}_q$
- Select the fold-specific threshold $\tau_b^{(q)}$ at FPR = 0.01 using benign samples in $\mathcal{F}_q$
- Derive the fold-specific rescue width δ_q from the benign fraction ϵ_{res}
- $\mathcal{R}_{\text{OOF}}^{(q)} \leftarrow \left\{(x_i, y_i) \in \mathcal{F}_q : s_i^{(q)} < \tau_b^{(q)} \wedge 0 < \tau_b^{(q)} - s_i^{(q)} \leq \delta_q\right\}$
- **foreach** $(x_i, y_i) \in \mathcal{R}_{\text{OOF}}^{(q)}$ **do**
  - $z_i \leftarrow y_i$
  - $\mathcal{D}_{res} \leftarrow \mathcal{D}_{res} \cup \{(x_i, z_i)\}$

`// Step 2: Train the final baseline and auxiliary rescue detectors`

Train the baseline detector $f : \mathbb{R}^{2568} \to \mathbb{R}$ on $\mathcal{D}_{train}$ using Θ_f

Train the auxiliary rescue detector $g : \mathbb{R}^{2568} \to [0, 1]$ on $\mathcal{D}_{res}$

`// Step 3: Generate cross-conformal validation values`

Compute $s_b(x) = f_{\text{raw}}(x)$ and $r(x) = g(x)$ for all $x \in \mathcal{D}_{val}$

Partition $\mathcal{D}_{val}$ into five stratified folds $\{\mathcal{V}_1, \ldots, \mathcal{V}_5\}$

foreach $k \in \mathcal{K}$ **do**
- Initialize cross-conformal values $p_1^{(k)}(x)$ for all baseline-negative $x \in \mathcal{D}_{val}$
- **for** $q \leftarrow 1$ **to** 5 **do**
  - $\mathcal{V}_{cal}^{(q)} \leftarrow \mathcal{D}_{val} \setminus \mathcal{V}_q$
  - Construct k rescue-score quantile contexts using baseline-negative samples in $\mathcal{V}_{cal}^{(q)}$
  - Construct context-specific and global class-conditional nonconformity distributions using baseline-negative true negatives and false negatives in $\mathcal{V}_{cal}^{(q)}$
  - **foreach** *baseline-negative* $x \in \mathcal{V}_q$ **do**
    - Assign x to its rescue-score context
    - Compute $p_1^{(k)}(x)$ using calibration distributions constructed without $\mathcal{V}_q$
- `// Step 4: Select the gate and rescue operating point for context size` k
- **foreach** *candidate conformal threshold* α **do**
  - $\mathcal{G}_{k,\alpha} \leftarrow \{x \in \mathcal{D}_{val} : s_b(x) < \tau_b \wedge p_1^{(k)}(x) > \alpha\}$
  - **if** $\text{TN}(\mathcal{G}_{k,\alpha}) \leq B_{\text{gate}}$ **then**
    - Select $\tau_r^{(k,\alpha)}$ that maximizes recovered validation false negatives subject to $\text{FP}_{add} \leq B_{\text{FP}}$
- For the current k select $(\alpha_k, \tau_{r,k})$ by maximum FN recovery, fewer additional FP, fewer gated TN, smaller gate, and higher α

`// Step 5: Select and freeze the final adaptive configuration`

Among the configurations selected for all $k \in \mathcal{K}$, choose $(k^*, \alpha^*, \tau_r^*)$ by maximum FN recovery, fewer additional FP, fewer gated TN, smaller gate, smaller k, and higher α

Refit the k^*-context conformal calibrator $\mathcal{C}^*$ using the complete validation split

Freeze f, g, τ_b, $\mathcal{C}^*$, k^*, α^*, and τ_r^*

`// Step 6: Evaluation on the standard test set`

foreach $(x, y) \in \mathcal{D}_{test}$ **do**
- $\hat{y} \leftarrow$ CGUARD_Decision(x)
- Update CM_{test}

`// Step 7: Evaluation on the challenge-only set`

foreach $x \in \mathcal{D}_{ch}$ **do**
- $\hat{y} \leftarrow$ CGUARD_Decision(x)
- Update FN_{ch} and TP_{ch}

$\mathsf{ER}_{ch} \leftarrow \mathsf{FN}_{ch}/|\mathcal{D}_{ch}|$

`// Step 8: Evaluation on the full challenge set (benign + evasive malware)`

$\mathcal{D}_{ch}^{full} \leftarrow \mathcal{D}_{test}^{benign} \cup \mathcal{D}_{ch}$

foreach $(x, y) \in \mathcal{D}_{ch}^{full}$ **do**
- $\hat{y} \leftarrow$ CGUARD_Decision(x)
- Update CM_{ch}

https://doi.org/10.3390/fi18080431

Let $\mathcal{A}_{1,j}$ denote the set of malware class nonconformity scores obtained from validation false negatives assigned to context j, and let $n_{1,j} = |\mathcal{A}_{1,j}|$. For a baseline-negative sample assigned to context $j(x)$, the malware class conformal compatibility value is computed as

$$p_1(x) = \frac{1 + \left|\left\{a \in \mathcal{A}_{1,j(x)} : a \geq a_1(x)\right\}\right|}{n_{1,j(x)} + 1}. \tag{2}$$

A high value of $p_1(x)$ indicates that the sample is compatible with the baseline-score behavior observed for validation false negatives in the same rescue-score context. When the number of class-specific calibration samples in a local context is below the predefined minimum, the corresponding global class conditional calibration distribution is used as a fallback.

The number of contexts k, conformal threshold α, and rescue threshold τ_r are selected on the validation split applying five-fold cross-conformal evaluation. For each candidate context configuration, baseline-negative validation samples receive conformal compatibility values based on calibration data that omit the corresponding evaluation fold. The chosen operating point maximizes the recovery of false negatives while adhering to the predefined additional false-positive budget and gated true-negative constraint. Following this selection, the final conformal calibrator is refit on the full validation split. The resulting context boundaries, class-conditional nonconformity distributions, global fallback distributions, and the selected values of α and τ_r are frozen for subsequent held-out evaluation.

At inference time, for an input sample x, C-GUARD proceeds as follows:

- If $s_b(x) \geq \tau_b$, the sample is classified as malware without further intervention.
- If $s_b(x) < \tau_b$, the rescue score $r(x)$ is computed and used to assign the sample to its frozen rescue-score context.
- The frozen conformal calibrator is applied to obtain the malware-class compatibility value $p_1(x)$.
- If $p_1(x) \leq \alpha$, the baseline benign prediction is kept.
- If $p_1(x) > \alpha$, the sample is admitted into the adaptive gate and flips to malware only when $r(x) \geq \tau_r$.

These conditions can be summarized by the following C-GUARD decision rule:

$$\hat{y}_{CG}(x) = \begin{cases} 1, & s_b(x) \geq \tau_b, \\ 1, & s_b(x) < \tau_b \wedge p_1(x) > \alpha \wedge r(x) \geq \tau_r, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

Algorithm 1 summarizes the complete training, validation-selection, and frozen inference procedures. It distinguishes three-fold out-of-fold rescue-data construction from five-fold cross-conformal gate selection and makes explicit that the standard test and challenge sets are not used for model training, calibration, or parameter selection.

C-GUARD preserves baseline-positive decisions, and it augments a selective rescue mechanism that operates on the baseline-negative samples admitted by the adaptive conformal gate. By integrating out-of-fold rescue learning, validation-only adaptive conformal gating, and explainability-informed diagnostic experiments, C-GUARD targets identified baseline failure regions while bounding the additional false positives.

4. Empirical Evaluation and Discussion

4.1. Dataset

EMBER2024 (Feature Version 3) [3] dataset, a large-scale, temporally structured benchmark for evaluating static malware detection systems is used to evaluate the proposed

method in this work. EMBER2024 includes a challenge set (evasive malware) in addition to the standard training and testing sets. The training set consists of weekly snapshots spanning from September 2023 to September 2024. The test set covers subsequent weekly snapshots from September 2024 to December 2024. The challenge set spans the same temporal range as the training and testing set. The challenge set in EMBER2024 consists of files that were not detected by any antivirus product of VirusTotal at the time of their initial scan on VirusTotal. These files were subsequently re-scanned at least 30 days later, at which point they were detected by a sufficient number of independent Anti-Virus (AV) engines and consequently labeled as malicious according to the dataset's labeling methodology. Notably, files included in the challenge set do not appear in the EMBER2024 training or test sets. Furthermore, near-duplicate files from the same week of data collection were explicitly excluded to prevent redundancy and information leakage.

Unlike controlled adversarial benchmarks, the EMBER2024 challenge set does not provide details about the specific evasion mechanism, attacker knowledge, or transformation strategies for each challenge sample. Therefore, C-GUARD is evaluated as an evasion-technique-agnostic detection defense for improving the detection of real-world evasive malware that initially bypassed antivirus detection.

The original EMBER2024 corpus contains heterogeneous file formats, including Windows PE binaries and non-PE artifacts (e.g., PDFs, ELF binaries, and APKs). Since the objective of this work is static Windows malware detection under a binary classification setting, we restrict our analysis exclusively to Windows PE files (Win32, Win64, .NET, and Dot_Net). Non-PE files were excluded. File labels were assigned based on AV detection obtained from VirusTotal. Files that remain undetected by all AV products after being re-scanned at least 30 days following their first submission to VirusTotal are labeled as benign (denoted as '0'). In contrast, files that are detected as malware by five or more independent AV products are labeled as malicious (denoted as '1'). The EMBER2024 dataset was downloaded using the official dataset interface, with the `file_type` parameter restricted to PE files.

A key characteristic of the EMBER2024 dataset is that the same samples appeared duplicated due to periodic re-scanning on VirusTotal. This results in duplicated hashes with potentially different timestamps. To address this issue, a hash-based de-duplication strategy was applied to each JSONL file independently. For each JSON file, the following steps were performed:

- Samples were grouped by their cryptographic hash (SHA-256, with fallbacks to SHA-1 or MD5 when necessary).
- If multiple records exist with the same hash, only the record with the most recent `last_analysis_date` was retained. All older scan records were discarded.

After PE-only filtering and de-duplication, the final distribution of training and test set samples exactly matched the distribution reported in EMBER2024 [3]. The training set was further divided into a training and validation subset using a stratified 90:10 split. The distribution of the samples used in C-GUARD is reported in Table 1.

Table 1. Distribution of PE samples in the training, validation, test, and challenge set.

Split	Benign	Malware	Total
Training Set	1,053,000	1,053,000	2,106,000
Validation Set	117,000	117,000	234,000
Test Set	270,000	270,000	540,000
Challenge Set	0	4868	4868

The training, validation, and test set are perfectly balanced, containing equal numbers of benign and malicious samples, whereas the challenge set consists exclusively of malicious files, enabling a focused evaluation of false negatives and evasion behavior.

4.2. Implementation Details

All experiments were implemented in Python 3.11. Due to the large scale of the dataset, the baseline decision model was implemented using LightGBM (v4.6.0), a gradient-boosted decision tree framework. The hyperparameter optimization was performed with Optuna (v4.6.0) [35], an open-source Python library, using the Tree-structured Parzen Estimator (TPE) algorithm [36]. The Optuna search was performed across 50 trials, and each trial trained a LightGBM model for up to 2000 boosting rounds with GPU acceleration. The hyperparameter search space explored during optimization is reported in Table 2. The objective, boosting type, execution device, tree learner, and maximum bin count were fixed to binary, gbdt, GPU, serial, and 255, respectively. The final baseline LightGBM model was retrained from scratch using the optimized parameter set on the training split. The validation split was used for baseline-threshold selection at a false-positive rate of 1% and adaptive-gate configuration. The resulting frozen raw-score threshold was $\tau_b = -0.12616331223988325$, corresponding to a sigmoid probability threshold of 0.4685009422114843.

Table 2. Hyperparameter search space used for Optuna-based optimization and selected configuration for the baseline LightGBM detector.

Hyperparameter	Search Range	Selected Value
Learning rate (η)	$[0.005,\ 0.05]$ (log-uniform)	0.045858
Number of leaves	$[128,\ 2048]$	1418
Maximum tree depth	$[-1,\ 20]$	12
Feature fraction	$[0.5,\ 1.0]$	0.905817
Bagging fraction	$[0.5,\ 1.0]$	0.535913
Bagging frequency	$[1,\ 5]$	3
Minimum data in leaf	$[50,\ 300]$	298
ℓ_1 regularization (λ_1)	$[0.0,\ 5.0]$	3.040481
ℓ_2 regularization (λ_2)	$[0.0,\ 5.0]$	0.606780
Minimum gain to split	$[0.0,\ 0.3]$	0.252338

The rescue training set was produced using three-fold stratified out-of-fold scoring on the training split. For each iteration, a fold-specific baseline detector was trained on 1,404,000 samples and scored the held-out fold of 702,000 samples. The fold-specific threshold corresponding to a 1% false-positive rate was applied, and a near-threshold rescue region was defined using the benign exposure fraction ϵ_{res}, used to derive the fold-specific score-distance width δ_q defining the out-of-fold rescue region. Notably, ϵ_{res} controls the benign exposure of the failure focused out-of-fold rescue region. The value of ϵ_{res} was selected through validation split experiments. An initial broad sweep examined $\epsilon_{\text{res}} \in \{0.0025, 0.005, 0.0075, 0.010, 0.015, 0.020\}$ across nine internal baseline FPR allocations from 0.80% to 1.00%. The smallest tested value, $\epsilon_{\text{res}} = 0.0025$, provided the largest false-negative recovery within every internal allocation while exposing the fewest benign samples. A subsequent focused refinement evaluated $\epsilon_{\text{res}} \in \{0.002, 0.005, 0.010\}$ at internal baseline allocations of 0.98%, 0.95%, and 0.90%, where $\epsilon_{\text{res}} = 0.002$ achieved the highest recovery at all three allocations, as summarized in Table 3. We therefore fixed $\epsilon_{\text{res}} = 0.002$ for out-of-fold rescue-region construction.

Table 3. Validation-only refinement of the rescue-region benign-exposure fraction. Reported recovery is relative to the corresponding internal baseline allocation.

Internal FPR	ϵ_{res}	Region TN	Region FN	Region Size	Recovered FN
0.98%	0.002	232	184	416	14
0.98%	0.005	580	382	962	8
0.98%	0.010	1159	616	1775	10
0.95%	0.002	232	210	442	45
0.95%	0.005	580	412	992	34
0.95%	0.010	1159	655	1814	34
0.90%	0.002	232	243	475	100
0.90%	0.005	580	465	1045	73
0.90%	0.010	1160	716	1876	58

The broader 9×6 exploratory sweep preceding this refinement is reported in Appendix A.1 (Table A1). Using the fixed $\epsilon_{\text{res}} = 0.002$, aggregation of eligible held-out true-negative and false-negative samples produced the final rescue-training set of 4297 samples, comprising 2106 true negatives and 2191 false negatives. The auxiliary rescue detector was implemented using LightGBM and trained on the rescue training set using 2568 EMBER features. Its hyperparameters were optimized through 40 Optuna trials using three-fold stratified cross-validation on the rescue training set. The selected configuration is reported in Table 4.

Table 4. Optimized hyperparameters of the auxiliary rescue detector. Continuous values are rounded to six decimal places.

Hyperparameter	Selected Value
Objective	`binary`
Evaluation metric	`binary_logloss`
Boosting type	`gbdt`
Device type	CPU
Learning rate	0.032211
Number of leaves	23
Maximum tree depth	6
Minimum data in leaf	25
Feature fraction	0.646072
Bagging fraction	0.683181
ℓ_1 regularization (λ_1)	0.000127
ℓ_2 regularization (λ_2)	0.116569
Minimum gain to split	0.199674
Random seed	42

Adaptive conformal gate selection was performed on the validation split using five-fold cross-conformal evaluation, as defined in Algorithm 1. The analysis included only baseline-negative validation samples, yielding 117,216 operational samples at the 1% baseline operating point, including 115,830 true negatives and 1386 false negatives. Candidate context configurations employed $k \in \{1, \ldots, 10\}$ rescue-score quantile bins. When a context-specific class calibration set contained fewer than $n_{\min} = 30$ observations, the corresponding global class-conditional distribution was used as a fallback. For each k, candidate conformal thresholds α were generated from the distinct cross conformal malware compatibility decision boundaries observed among baseline negative validation samples. The corresponding adaptive gate was defined as

$$G_{k,\alpha} = \left\{ x \in \mathcal{D}_{\text{val}} \;\middle|\; s_b(x) < \tau_b \wedge p_1^{(k)}(x) > \alpha \right\}. \tag{4}$$

A candidate gate was considered feasible only when $TN(G_{k,\alpha}) \leq B_{\text{gate}} = 234$. $B_{\text{gate}} = 234$ denotes the maximum number of validation true negatives permitted to enter the adaptive gate.

For each feasible (k, α) configuration, the rescue threshold τ_r was selected over the available auxiliary score decision groups of the gated samples to maximize recovered validation false negatives subject to an additional false-positive budget of $B_{\text{FP}} = 24$. Within each candidate k, feasible (α, τ_r) configurations were ranked by maximum false-negative recovery, fewer additional false positives, fewer gated true negatives, smaller total gate size, and higher α. The best configuration from each k was then compared across k using the same criteria, with smaller k preferred before the final higher-α tie break. When multiple rescue thresholds achieved the same false-negative recovery, the threshold producing fewer additional false positives was preferred, followed by the higher threshold. The selected configuration used $k^* = 9$, $\alpha^* = 0.9173456680305996$, and $\tau_r^* = 0.5597061541865259$. On the validation split, this configuration gated 117 true negatives and 126 false negatives, recovered 57 false negatives, and introduced 24 additional false positives. After configuration selection, the conformal calibrator was refit on the complete validation split. The resulting rescue score context boundaries, context specific and global class conditional nonconformity distributions, α^*, and τ_r^* were frozen. The baseline and rescue models were not retrained during this refitting step. During inference, baseline malware predictions were unchanged, while the baseline goodware predictions satisfying both the conformal gate and rescue score conditions were promoted to the malware class.

To determine whether C-GUARD benefit is limited to the primary 1% operating point, additional nominal baseline validation false-positive rate targets are considered, including 0.10%, 0.25%, 0.50%, 0.75%, and 1.00%. For each target, the baseline threshold is selected using the validation split. Subsequently, the adaptive parameters k and α and the auxiliary rescue threshold τ_r are selected independently under the same constraints as in the primary experiment: at most 24 additional validation false positives and at most 234 gated validation true negatives. The auxiliary rescue detector remains fixed throughout the analysis. For each operating point, a threshold-relaxation baseline is constructed to evaluate whether the same validation false-positive budget can be utilized by lowering the baseline decision threshold. The threshold-relaxation is selected on the validation set to match the final C-GUARD false-positive count whenever the baseline score values permit an exact match and is then fixed for evaluation on the standard test and challenge sets. Thus, the comparator is FP-matched to C-GUARD on validation, although its realized held-out false-positive rate may differ after the threshold is fixed. Notably, no standard test or challenge labels are used for parameter selection.

To quantify statistical reliability at the 1% operating point, we use 5000 paired bootstrap replicates [37] to estimate 95% confidence intervals for the differences between the baseline and C-GUARD. McNemar tests [38] are additionally used for paired comparisons. The analyses are performed separately for standard test benign samples, standard test malware, and challenge malware.

4.3. Analysis of the Results

Table 5 summarizes the performance of the baseline and the proposed C-GUARD framework on both the standard test set and the challenge set. The discussion focuses primarily on changes in false negatives, true positives, recall, false-positive rate, and false-negative rate, as these measures directly reflect the trade-off between recovering missed malware and preserving benign classification performance.

On the standard test set, both models achieve near-identical and near-perfect performance. Nevertheless, C-GUARD provides a small but consistent improvement in malware detection. Specifically, the number of false negatives is reduced from 5059 to 4881, while true positives increase from 264,941 to 265,119. This corresponds to the successful recovery of 178 malware samples that were misclassified as benign by the baseline detector. Accordingly, recall increases from 0.9813 to 0.9819, while the false-negative rate decreases from 0.0187 to 0.0181. These improvements are accompanied by a small increase in false positives from 3188 to 3270, resulting in a false-positive rate increase from 0.0118 to 0.0121. Thus, C-GUARD recovers additional malware at the cost of 82 additional benign samples being classified as malicious. C-GUARD improves malware detection while preserving the overall strong classification performance of the baseline. These results demonstrate that C-GUARD preserves the strong standard-test performance of the baseline while selectively recovering additional false negatives from the baseline-negative region.

A different behavior emerges under the full challenge evaluation setting, where benign test samples are jointly evaluated with the 4868 evasive malware samples. In this more demanding scenario, the baseline detector misclassifies 1408 evasive malware samples, corresponding to a recall of 0.7108 and a false-negative rate of 0.2892. This result shows that, despite its strong performance on the standard test set, the baseline remains vulnerable to a substantial subset of real-world evasive malware. C-GUARD reduces the number of false negatives from 1408 to 1391 and increases the number of true positives from 3460 to 3477. This shows the recovery of 17 evasive malware samples that bypass the optimized baseline detector. As a result, recall increases from 0.7108 to 0.7143, while the false-negative rate decreases from 0.2892 to 0.2857. Although the aggregate numerical change is modest, the improvement is operationally significant. Even a single missed evasive malware sample may be sufficient to compromise a host, steal sensitive information, establish persistence, disrupt services, or help spread malware further within a network. So, each recovered false negative helps prevent a potentially serious security issue. The recovery observed under the full challenge evaluation is associated with 82 additional false positives, increasing from 3188 to 3270. However, this rise originates from a much larger set of 270,000 benign samples, resulting in only a marginal increase in the false-positive rate from 0.0118 to 0.0121. This trade-off is consistent with the design of C-GUARD, which preserves all baseline malware class decisions and allows only a carefully selected subset of baseline benign class decisions to be promoted to the malicious class. The observed robustness gain is achieved through targeted corrections applied only to samples admitted by the adaptive conformal gate and subsequently validated by the auxiliary rescue detector. This approach yields a controlled improvement in robustness with minimal effect on the overall benign error rate.

Table 5. Performance evaluation of the baseline and C-GUARD evaluated on the standard test set and full challenge set (Benign test set + Evasive challenge malware). The best results are underlined.

Evaluation Set	Model	TN	FP	FN	TP	Recall	FPR	FNR
Standard Test Set	Baseline	266,812	3188	5059	264,941	0.9813	0.0118	0.0187
	C-GUARD	266,730	3270	4881	265,119	0.9819	0.0121	0.0181
Full Challenge Set	Baseline	266,812	3188	1408	3460	0.7108	0.0118	0.2892
	C-GUARD	266,730	3270	1391	3477	0.7143	0.0121	0.2857

To explicitly quantify robustness against evasive malware, we compute the evasion rate, defined as the fraction of evasive malware samples that successfully bypass the detector:

$$\text{Evasion Rate} = \frac{\text{Number of misclassified evasive malware samples}}{\text{Total number of evaluated evasive malware samples}}. \quad (5)$$

Table 6 reports the evasion rate computed exclusively on the challenging evasive malware samples. The baseline model misclassifies 1408 out of 4868 evasive malware samples, resulting in an evasion rate of 0.2892. This evasion rate highlights the vulnerability of the baseline detector to real-world evasive malware despite its strong performance on the standard test set. In contrast, C-GUARD reduces the number of misclassified evasive samples to 1391. This reduction corresponds to the successful recovery of 17 evasive malware samples that bypass the baseline detector, resulting in a decrease of 0.0035 in the evasion rate. Although the overall improvement is modest, however, this improvement is obtained against a strong optimized baseline and under an evasion-technique-agnostic setting where the evasion methods of the challenge samples are unknown.

Table 6. Challenge-set evasion rate of the baseline detector and C-GUARD. Best results are underlined.

Evaluation Set	Model	Evasive Malware	Evasion Rate
Challenge Set (Evasive Only)	Baseline	1408/4868	0.2892
	C-GUARD	<u>1391/4868</u>	<u>0.2857</u>

Overall, the aggregate performance metrics, evasion-rate analysis, and confusion matrix counts provide a coherent and complementary view of the effectiveness of C-GUARD. On the standard test set, C-GUARD recovers 178 additional malware samples while maintaining baseline detection performance and recovers 17 evasive malware samples under the challenge setting. This behavior results in an increase in true positives and recall while decreasing the false-negative rate and evasion rate. Collectively, these results demonstrate that C-GUARD enhances robustness in a targeted and controlled manner, strengthening malware detection against challenging evasive malware and standard malware with a small quantified increase in false positives.

To assess the statistical reliability of the results at 1% baseline operating point, Table 7 reports paired bootstrap confidence intervals and McNemar tests for standard test recall, standard test FPR, and challenge set detection rate.

Table 7. Paired statistical reliability analysis at the 1% baseline operating point. Differences are computed as C-GUARD minus baseline. The 95% confidence intervals are obtained from 5000 paired bootstrap replicates, and p-values are from exact McNemar tests. pp denotes percentage points.

Evaluation	Baseline	C-GUARD	Difference (pp)	95% Bootstrap CI (pp)	McNemar p
Standard-test recall	98.1263%	98.1922%	+0.0659	[0.0567, 0.0752]	5.22×10^{-54}
Standard-test FPR	1.1807%	1.2111%	+0.0304	[0.0237, 0.0370]	4.14×10^{-25}
Challenge detection rate	71.0764%	71.4256%	+0.3492	[0.1849, 0.5341]	1.53×10^{-5}

On the standard test set, the recall improvement corresponds to 178 baseline false negatives recovered by C-GUARD, while the FPR increase corresponds to 82 additional false positives. On the evasive challenge set, C-GUARD additionally detects 17 malware samples missed by the baseline. The paired confidence intervals exclude zero for all three differences, and the McNemar tests indicate statistically significant paired changes, supporting the reliability of both the malware detection gains and the associated increase in false positives.

4.4. Performance Across Lower-FPR Operating Points

The primary evaluation considers a 1% baseline operating point at the validation split. To examine C-GUARD effectiveness under stricter operating conditions, we additionally evaluate nominal baseline FPR targets of 0.10%, 0.25%, 0.50%, 0.75%, and 1.00%. For

each target, the baseline threshold and adaptive C-GUARD configuration are selected independently on validation split, while the auxiliary rescue detector remains fixed. We compare C-GUARD with the threshold-relaxation baseline under the same validation false-positive count. As shown in Table 8, C-GUARD produces fewer false negatives at every operating point. At the strictest 0.10% target, C-GUARD yields 5991 false negatives compared with 6190 for the threshold-relaxation baseline, corresponding to 199 additional malware detections at the same 141 false positives. The corresponding reductions are 133, 80, 36, and 39 false negatives at the 0.25%, 0.50%, 0.75%, and 1.00% operating points, respectively. These results show that C-GUARD improvement is not simply due to lowering the baseline decision threshold.

The validation-selected configurations are further evaluated on the standard test and evasive challenge set. C-GUARD improves malware recovery at all five operating points, recovering 3107, 1103, 449, 218, and 178 standard-test false negatives while introducing 116, 120, 99, 85, and 82 additional false positives, respectively. On the evasive challenge set, the corresponding recoveries are 172, 115, 41, 26, and 17 samples. Thus, the benefit of selective rescue is not confined to the primary 1% operating point and remains observable across stricter low-FPR regimes. Detailed validation-selected configurations and frozen held-out results, including realized FPR and the validation-matched threshold-relaxation baseline, are reported in Appendix B.

Table 8. Performance evaluation across low-FPR operating points on validation, standard test, and challenge set. The threshold-relaxation baseline is FP-matched to C-GUARD on validation. ΔFP denotes additional standard-test false positives introduced by C-GUARD.

Nominal	Validation FN		Standard Test			Challenge
FPR	C-GUARD	Threshold-Relaxation	Baseline FN	C-GUARD FN	ΔFP	Recovered
0.10%	5991	6190	20,710	17,603	116	172
0.25%	3635	3768	12,666	11,563	120	115
0.50%	2230	2310	8076	7627	99	41
0.75%	1674	1710	6179	5961	85	26
1.00%	1329	1368	5059	4881	82	17

4.5. Feature Representation and Auxiliary Detector Ablation

Static malware detectors trained on large-scale feature representations often achieve near-saturated performance under standard evaluation settings, yet their effectiveness can degrade sharply when exposed to evasive malware. In our preliminary experiments on the EMBER2024 dataset, a LightGBM classifier trained on the full feature space achieved a high detection rate at a fixed operating point on the standard test set but exhibited a substantial increase in false negatives on an additional challenge set composed exclusively of evasive malware samples. This discrepancy motivated a deeper investigation of baseline failure factors; particularly, we examined whether misclassified malware (false negative) samples keep sufficient evidence to support selective recovery. The investigation was performed on the validation set, which showed that a subset of baseline-negative malware samples keeps malicious evidence despite being misclassified as benign by the baseline. This observation motivated an auxiliary rescue detector to intervene on the gated baseline-negative samples. The auxiliary rescue detector is trained on a small and failure-focused rescue set. We focus on understanding rescue detector recovery of gated baseline-negative samples on EMBER full feature space and reduced feature space derived using baseline failure patterns.

Specifically, we investigate the following questions:

- Can failure pattern feature subsets produced from the validation split match or compete with the results achieved on EMBER full feature space?
- Are alternative auxiliary rescue detector architectures more effective than LightGBM?
- Can complementary evidence from multiple reduced feature spaces collectively replace the EMBER full feature space?
- Does removing misleading features that push baseline false negatives toward the benign class improve rescue detector performance?

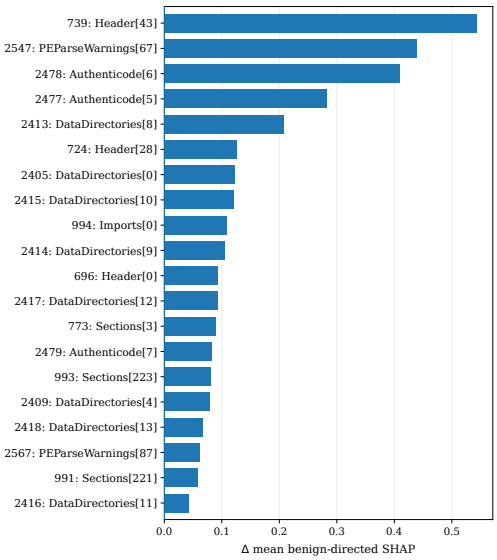
4.5.1. Failure Diagnosis

To address these questions, the optimized baseline is evaluated on the validation split and produces 115,830 true negatives, 1170 false positives, 1386 false negatives, and 115,614 true positives. Baseline-negative samples (true negatives and false negatives) are further divided into two subgroups based on their position relative to the baseline decision boundary using the fixed Platt margin gate calibrated from the near-threshold benign validation distribution. Samples falling within the gate are grouped as gated-negatives, while the samples falling outside the gate are grouped as non-gated negatives. The gated-negatives group comprises 234 true negatives and 192 false negatives, while the non-gated negatives group comprises 115,596 true negatives and 1194 false negatives.

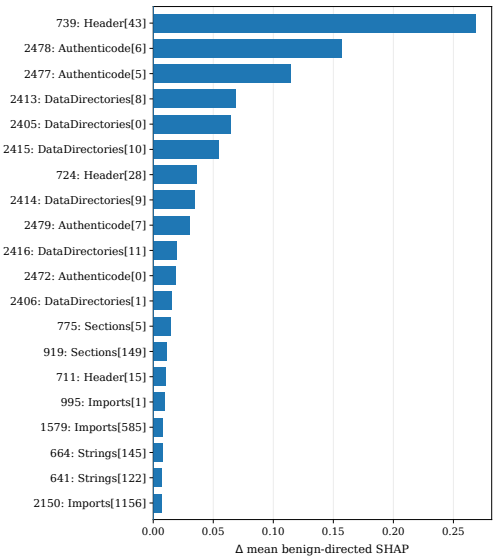
To mitigate confounding by baseline confidence, each of the 1,386 false-negative samples was matched to a non-gated true negative with a closely aligned calibrated probability. The resulting pairs exhibited strong score agreement, with a median absolute probability difference of approximately 3.05×10^{-5}, Pearson and Spearman correlations of 0.9797 and 0.9744, respectively, and 85.8% of the pairs differing by no more than 0.001. The resulting diagnostic dataset comprised all 234 gated true negatives, 192 gated false negatives, 1194 non-gated false negatives, and 1170 false positives, together with 1386 margin-matched non-gated true negatives, 1386 randomly sampled non-gated true negatives, and 1386 randomly sampled true positives. These cohorts provided gate-local, confidence-matched, benign-reference, and correctly detected malware comparisons, yielding a total of 6948 validation samples (diagnostic dataset).

Feature contributions to the model decision are computed in the raw-score space using SHAP [8] on the diagnostic dataset and the optimized baseline detector. Notably, the contributions are obtained on the full EMBER feature space. For each sample, the SHAP contributions with the expected value term reconstructed the corresponding baseline raw score to numerical precision, with a maximum additivity error of approximately 8.44×10^{-14}.

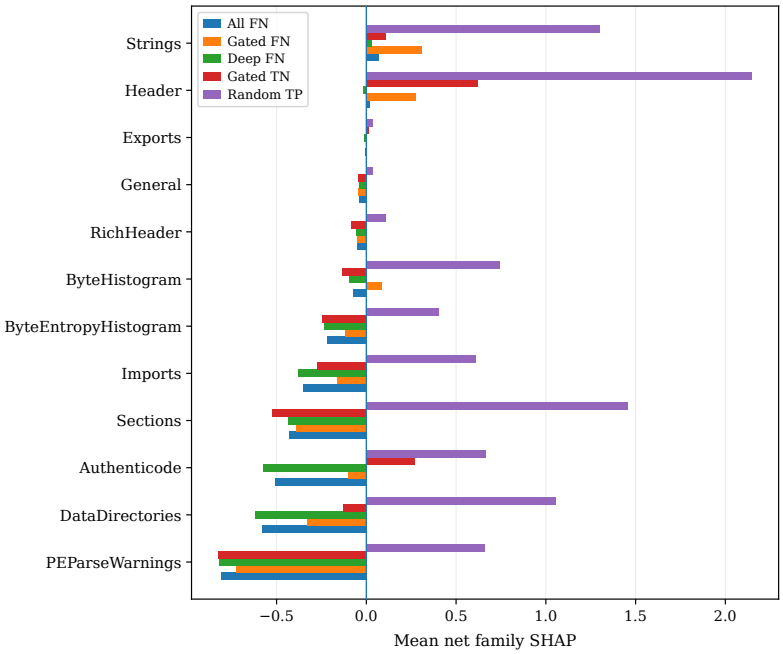
The SHAP comparisons of all false negatives with correctly detected malware and the gated false negatives with gated true negatives show that false-negative malware exhibited stronger benign-directed attribution. The dominant contribution families are Header, Authenticode, DataDirectories, PEParseWarnings, Sections, and Imports. The most prominent individual features are Header_43, Authenticode_5–6, DataDirectories_8, and PEParseWarnings_67. These findings show that baseline failures are associated with consistent attribution patterns across multiple EMBER feature families. However, the analysis does not establish whether removing these features or feature families would improve auxiliary detection. Figure 2 summarizes the attribution patterns.



(**a**) Benign-directed attribution in false negatives relative to correctly detected malware



(**b**) Gate-local attribution differences between gated false negatives and gated true negatives.



(**c**) Directional SHAP profiles across EMBER feature families and diagnostic cohorts.

Figure 2. Baseline SHAP analysis on validation split. (**a**) Top-20 features with increased benign directed attribution in false negatives relative to correctly detected malware. (**b**) Top-20 gate-local attribution differences between gated false negatives and gated true negatives. (**c**) Directional SHAP profiles across EMBER feature families and diagnostic cohorts.

4.5.2. XAI-Grounded Representation Construction and Fixed-Gate Screening

We further examined the stability of attribution patterns under resampling by using 2000 bootstrap replicates with 95% confidence intervals across four validation split comparisons: gated false negatives versus gated true negatives, all false negatives versus randomly sampled true positives, all false negatives versus margin-matched non-gated true negatives,

and deep false negatives versus gated false negatives. We consider a feature as stable if the upper confidence bound of the signed-SHAP difference is below zero and the lower confidence bound of the benign-directed attribution difference is above zero. This analysis produces three candidate feature representations, including 95-feature two-contrast consensus (Strict95), 16-feature three-contrast consensus (Strong16), and 31-feature gate-local consensus (GateLocal31). The strict and strong consensus sets are dominated by the Authenticode, DataDirectories, and Header feature families. In addition, we produces family-based feature representations from the EMBER feature families that exhibit stable benign-directed attribution in the corresponding comparisons. These constructed feature representations include Core3, comprising Authenticode, DataDirectories, and Header families; GateLocal, comprising Header and Authenticode families; MatchedResidual, comprising DataDirectories and Authenticode families; and FailureDepth, comprising ByteHistogram, Strings, Header, Imports, DataDirectories, and Authenticode families. We also produces complement feature representations by removing identified features or feature families from full EMBER feature representation. The complement feature representations are as follows: Core3-Excluded, obtained by removing Core3 feature representation families; FailureDepth-Excluded, obtained by removing FailureDepth feature representation families; Strict95-Excluded and Strong16-Excluded, obtained by removing the corresponding feature-level candidates.

Each auxiliary LightGBM model is trained on the rescue dataset (4297 samples) using exactly one single feature representation under the same parameter configuration reported in Table 4, evaluated on fixed gated baseline negatives (426 samples) with a budget of maximum 24 additional false positives beyond the baseline 1% FPR operating point. The model trained with MatchedResidual feature representation achieved highest false-negatives recovery, with 22 additional false positives. The results are reported in Table 9.

Table 9. Auxiliary LightGBM feature representation screening under the fixed gate. The best recovery is underlined.

Representation	Features	Extra FP	Recovered FN
Full EMBER	2568	24	43
GateLocal31	31	23	43
Strict95	95	23	39
Strong16	16	23	37
MatchedResidual	42	22	<u>46</u>
Core3	116	23	45
GateLocal	82	24	43
FailureDepth	1831	21	41
Core3-Excluded	2452	23	39
FailureDepth-Excluded	737	21	32
Strict95-Excluded	2473	20	34
Strong16-Excluded	2552	23	37

4.5.3. Adaptive Feature-Representation Screening

Different feature representations may induce different optimal conformal contexts and rescue thresholds. Therefore, we extend our analysis to investigate feature-space-specific gate selection by allowing each feature space to independently select $k \in \{1, \ldots, 10\}$, the conformal threshold α, the adaptive gate, and the auxiliary decision threshold under a maximum budget of 24 additional false positives and a maximum of 234 gated true negatives. The auxiliary detector trained with the full feature representation achieved maximum recovery of 57 false negatives with $k = 9$ at 24 additional false positives. The auxiliary trained on FailureDepth feature representation achieved the same recovery but requires a larger gate, comprising 183 true negatives and 178 false negatives compared

to the full feature representation comprising 117 true negatives and 126 false negatives. The auxiliary trained with Core3 feature representation remained a competitive candidate. MatchedResidual achieved the strongest recovery under the fixed gate; however, it recovered 52 false negatives with 24 additional false positives under adaptive configuration. The results are summarized in Table 10. The results show that the auxiliary trained on full feature representation achieves the best performance overall by exposing fewer benign samples and fewer samples overall to auxiliary intervention under representation specific adaptive gating, and, therefore, this gating was selected for the final framework.

Table 10. Auxiliary LightGBM feature-representation screening under the representation-specific adaptive conformal protocol. The best recovery is underlined.

Representation	Features	k	Gated TN	Gated FN	Total Gate	Extra FP	Recovered FN
Full EMBER	2568	9	117	126	243	24	<u>57</u>
GateLocal31	31	8	103	110	213	23	50
Strict95	95	8	173	175	348	24	50
Strong16	16	10	115	120	235	24	41
MatchedResidual	42	1	225	183	408	24	52
Core3	116	4	158	164	322	23	55
GateLocal	82	7	141	134	275	23	43
FailureDepth	1831	4	183	178	361	24	<u>57</u>
Core3-Excluded	2452	8	131	126	257	23	42
FailureDepth-Excluded	737	8	154	146	300	22	41
Strict95-Excluded	2473	9	145	140	285	24	48
Strong16-Excluded	2552	6	233	216	449	24	47

4.5.4. Auxiliary Architecture Ablation Under Adaptive Gating

This analysis investigates alternative auxiliary architectures, including shallow Multilayer Perceptron (MLP) and a fixed equal-weight LightGBM-MLP ensemble under the adaptive conformal protocol. The MLP architecture consists of a single fully connected hidden layer with 256 ReLU units, followed by dropout with probability 0.284 and a single logit output. The network is trained using a learning rate of 2.26×10^{-4}, weight decay of 5.49×10^{-7}, batch size of 256, AdamW optimizer, class weighted binary cross-entropy, and gradient clipping at 5.0. The MLP configuration was selected by a 30-trial Optuna study conducted on the rescue dataset using nested three-fold evaluation with inner calibration for early stopping and threshold selection. The LightGBM auxiliary trained with full feature representation remained at the top by recovering 57 false negatives. Table 11 reports the representation–architecture comparison.

Table 11. Auxiliary-model architecture ablation under the adaptive conformal protocol. The best results are underlined.

Representation	Features	Architecture	k	Gated TN	Gated FN	Total Gate	Extra FP	Recovered FN
Full EMBER	2568	LightGBM	9	117	126	243	24	<u>57</u>
		MLP	7	28	35	63	24	35
		LightGBM-MLP	7	70	64	134	23	44
GateLocal31	31	LightGBM	8	103	110	213	23	50
		MLP	3	199	170	369	23	48
		LightGBM-MLP	1	225	183	408	24	43
Strict95	95	LightGBM	8	173	175	348	24	50
		MLP	10	42	49	91	24	46
		LightGBM-MLP	3	42	44	86	24	37
Strong16	16	LightGBM	10	115	120	235	24	41
		MLP	10	186	180	366	24	48
		LightGBM-MLP	8	101	100	201	23	42
Core3	116	LightGBM	4	158	164	322	23	55
		MLP	6	114	119	233	24	43
		LightGBM-MLP	3	144	135	279	23	46

Table 11. *Cont.*

Representation	Features	Architecture	*k*	Gated TN	Gated FN	Total Gate	Extra FP	Recovered FN
MatchedResidual	42	LightGBM	1	225	183	408	24	52
		MLP	9	124	120	244	24	50
		LightGBM-MLP	1	225	183	408	24	55
GateLocal	82	LightGBM	7	141	134	275	23	43
		MLP	4	102	97	199	24	40
		LightGBM-MLP	10	103	100	203	23	41
FailureDepth	1831	LightGBM	4	183	178	361	24	57
		MLP	7	218	191	409	24	45
		LightGBM-MLP	9	232	206	438	24	48
Core3-Excluded	2452	LightGBM	8	131	126	257	23	42
		MLP	4	48	48	96	23	37
		LightGBM-MLP	3	144	127	271	24	39
FailureDepth-Excluded	737	LightGBM	8	154	146	300	22	41
		MLP	9	89	79	168	24	37
		LightGBM-MLP	7	77	68	145	24	38
Strict95-Excluded	2473	LightGBM	9	145	140	285	24	48
		MLP	8	108	103	211	23	34
		LightGBM-MLP	10	145	136	281	24	51
Strong16-Excluded	2552	LightGBM	6	233	216	449	24	47
		MLP	10	66	66	132	24	34
		LightGBM–MLP	8	70	65	135	23	34

4.5.5. Cross-Representation Soft-Voting Ablation

We further evaluated cross-representation soft voting by restricting each auxiliary to one of the 11 reduced feature representations. Within each model family, the resulting prediction scores are averaged with equal weights. The LightGBM soft-voting ensemble (LightGBM-Ensemble) achieved the stronger result, recovering 47 false negatives with 23 additional false positives, whereas the corresponding MLP ensemble (MLP-Ensemble) recovered 37 with 24 additional false positives. Neither ensemble matched the 57 recoveries achieved by the single LightGBM auxiliary trained on the complete EMBER feature space. Table 12 summarizes these comparisons. Overall, the results confirm that a single Full EMBER LightGBM auxiliary provides the most effective rescue configuration.

Table 12. Cross-representation soft-voting ablation under the adaptive conformal protocol. Best recovery is underlined.

Candidate	Models	*k*	Gated TN	Gated FN	Total Gate	Extra FP	Recovered FN
LightGBM	1	9	117	126	243	24	57
LightGBM-Ensemble	11	4	173	172	345	23	47
MLP-Ensemble	11	7	72	62	134	24	37

The ablation results collectively support LightGBM utilizing full EMBER feature representation as an auxiliary detector within C-GUARD. Although the XAI-deriven reduced representations and alternative auxiliary architectures provided complementary recovery evidence, none surpassed the 57 false negatives recovered by the LightGBM auxiliary with full feature representation under the same adaptive conformal protocol and false-positive budget.

5. Conclusions and Future Work

This work analyzes why high-performing static Windows PE malware detectors fail under evasive manipulation, using an explainability-driven diagnostic workflow. By examining decision-margin behavior, directional SHAP attributions, matched baseline errors, and feature-space ablations, we identify attribution and margin differences between baseline true negatives and false negatives. However, the results also show that these differences do not define a universally superior reduced feature representation, indicating that feature pruning alone is insufficient to address the observed failure behavior.

Motivated by these findings, we propose C-GUARD, a context-adaptive conformal gated framework. C-GUARD preserves the full-feature baseline and selectively intervenes only on baseline-negative predictions. C-GUARD combines an out-of-fold-trained auxiliary rescue detector with a validation-only adaptive conformal gate. An asymmetric intervention rule allows for correcting baseline benign predictions while never overriding malware decisions. C-GUARD improves false-negative recovery under an explicitly constrained false-positive budget.

Experiments on EMBER2024 show that C-GUARD preserves overall performance of the baseline while selectively recovering additional malware samples. On the standard test set, it recovers 178 baseline false negatives at the cost of 82 additional false positives. Under the challenge setting, it recovers 17 evasive malware samples, reducing the evasion rate from 0.2892 to 0.2857, with only a marginal increase in the false-positive rate from 0.0118 to 0.0121. The ablation results further support the LightGBM with full EMBER feature representation as the rescue detector within C-GUARD. At the same time, the work highlights clear limitations. A substantial number of evasive samples remain undetected, particularly those confidently misclassified as benign. Moreover, the EMBER2024 challenge set does not specify the underlying evasion mechanisms or attacker knowledge, so the effectiveness of C-GUARD against particular adaptive or framework-aware attack strategies cannot be determined. Although the temporally separated EMBER2024 test set provides an initial evaluation under temporal variation, robustness under longer periods and broader distribution shifts remains to be established. Future work should investigate stronger representations and learning strategies for deeply missed evasive malware, evaluations against explicitly defined adaptive attacks, validation across additional datasets, time periods, and feature extractors. SHAP remains an offline diagnostic tool for understanding baseline failures and designing ablations rather than a runtime component of C-GUARD.

Author Contributions: Conceptualization, M.I.; methodology, M.I.; software, M.I. and F.F.; validation, M.I. and M.A.-E.; formal analysis, M.A.-E.; investigation, M.A.-E.; data curation, M.I. and F.F.; visualization, F.F.; writing—original draft preparation, M.I. and M.A.-E.; writing—review and editing, M.I. and M.A.-E.; supervision, G.P.; project administration, G.P.; funding acquisition, G.P. All authors have read and agreed to the published version of the manuscript.

Funding: This research was co-funded by the European Union—Next Generation EU: NRRP Initiative, Mission 4, Component 2, Investment 1.3—Partnerships extended to universities, research centers, companies, and research D.D. MUR n. 341 del 5.03.2022—Next Generation EU (PE00000014—"Security and Rights In the CyberSpace—SERICS"—CUP: H93C22000620001).

Data Availability Statement: The EMBER2024 dataset analyzed in this study is publicly available from its original distributors (https://github.com/FutureComputing4AI/EMBER2024/, accessed on 21 June 2026). Additional experimental artifacts are available from the corresponding author upon reasonable request, subject to applicable dataset licensing and security restrictions.

Acknowledgments: The authors wish to thank the anonymous reviewers for their helpful suggestions to improve this paper.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence;
ML	Machine Learning;
DL	Deep Learning;
PE	Portable Executable;
XAI	Explainable Artificial Intelligence;

SHAP	SHapley Additive exPlanations;
LightGBM	Light Gradient Boosting Machine;
TP	True Positive;
FN	False Negative;
FNR	False-Negative Rate;
FPR	False-Positive Rate;
LIEF	Library to Instrument Executable Formats;
AV	Anti-Virus;
TPE	Tree-structured Parzen Estimator;
MLP	Multilayer Perceptron.

Appendix A. Rescue-Region Selection and Adaptive-Gate Analysis

Appendix A.1. Rescue-Region Benign Exposure Sweep

This analysis evaluated the sensitivity of the preliminary rescue region to the benign exposure fraction ϵ_{res} on the validation split by evaluating six candidate fractions, $\epsilon_{\text{res}} \in \{0.0025, 0.005, 0.0075, 0.010, 0.015, 0.020\}$, across nine internal baseline false-positive rate allocations ranging from 0.80% to 1.00%. For each internal baseline threshold, the corresponding LightGBM raw-score distance width δ_{raw} was determined from the ϵ_{res} quantile of the internal baseline true-negative distance distribution. For each internal baseline allocation, the remaining false-positive budget up to the final 1% validation operating point was assigned to the auxiliary intervention. Table A1 summarizes all evaluated configurations. The smallest tested fraction, $\epsilon_{\text{res}} = 0.0025$, achieved the highest false-negative recovery at each internal baseline allocation in this broad search while exposing the smallest benign subset.

Table A1. Rescue-region sweep on validation split. δ_{raw} denotes the LightGBM raw score distance width induced by the corresponding benign exposure fraction ϵ_{res}. Recovered FN denotes false negatives recovered relative to the baseline operating at the corresponding internal FPR allocation; Final FN denotes the remaining validation false negatives after auxiliary intervention.

Internal FPR	ϵ_{res}	δ_{raw}	Region TN	Region FN	Region Size	Extra FP	Recovered FN	Final FN	τ_r
1.00%	0.0025	0.34155	290	235	525	0	1	1385	1.00000
1.00%	0.0050	0.63033	580	376	956	0	0	1386	1.00000
1.00%	0.0075	0.91752	869	526	1395	0	0	1386	1.00000
1.00%	0.0100	1.10943	1159	604	1763	0	0	1386	1.00000
1.00%	0.0150	1.47812	1738	748	2486	0	0	1386	1.00000
1.00%	0.0200	1.81724	2317	858	3175	0	0	1386	1.00000
0.98%	0.0025	0.34225	290	232	522	24	16	1389	0.99993
0.98%	0.0050	0.63426	580	382	962	24	8	1397	0.999999
0.98%	0.0075	0.92442	869	538	1407	24	10	1395	0.999999
0.98%	0.0100	1.12025	1159	616	1775	24	10	1395	0.999999
0.98%	0.0150	1.49334	1738	761	2499	24	10	1395	1.000000
0.98%	0.0200	1.83328	2318	876	3194	24	9	1396	1.000000
0.95%	0.0025	0.36557	290	256	546	59	46	1405	0.99245
0.95%	0.0050	0.65615	580	412	992	59	34	1417	0.99987
0.95%	0.0075	0.94844	870	572	1442	59	33	1418	0.99997
0.95%	0.0100	1.15865	1159	655	1814	59	34	1417	0.99998
0.95%	0.0150	1.52691	1739	798	2537	59	33	1418	1.00000
0.95%	0.0200	1.86610	2318	916	3234	59	21	1430	1.00000
0.92%	0.0025	0.37235	290	264	554	94	63	1427	0.96186
0.92%	0.0050	0.67451	580	435	1015	94	54	1436	0.99844
0.92%	0.0075	0.97911	870	595	1465	94	49	1441	0.99971
0.92%	0.0100	1.19690	1160	686	1846	94	49	1441	0.99986
0.92%	0.0150	1.56415	1739	828	2567	94	50	1440	0.99997
0.92%	0.0200	1.90526	2319	948	3267	94	44	1446	0.99999

Table A1. *Cont.*

Internal FPR	ϵ_{res}	δ_{raw}	Region TN	Region FN	Region Size	Extra FP	Recovered FN	Final FN	τ_r
0.90%	0.0025	0.40337	290	289	579	117	102	1427	0.84403
0.90%	0.0050	0.70428	580	465	1045	117	73	1456	0.99553
0.90%	0.0075	1.00369	870	618	1488	117	67	1462	0.99891
0.90%	0.0100	1.22967	1160	716	1876	117	58	1471	0.99972
0.90%	0.0150	1.60046	1740	862	2602	117	67	1462	0.99990
0.90%	0.0200	1.94120	2319	980	3299	117	56	1473	0.99997
0.87%	0.0025	0.42712	290	306	596	153	138	1432	0.59355
0.87%	0.0050	0.72947	580	484	1064	153	92	1478	0.98559
0.87%	0.0075	1.03265	870	642	1512	153	90	1480	0.99596
0.87%	0.0100	1.26095	1160	749	1909	153	77	1493	0.99868
0.87%	0.0150	1.63736	1740	891	2631	153	77	1493	0.99973
0.87%	0.0200	1.97763	2320	1014	3334	153	71	1499	0.99990
0.85%	0.0025	0.45462	291	319	610	176	161	1445	0.31999
0.85%	0.0050	0.76511	581	507	1088	176	116	1490	0.96356
0.85%	0.0075	1.06534	871	659	1530	176	108	1498	0.99237
0.85%	0.0100	1.30419	1161	780	1941	176	90	1516	0.99747
0.85%	0.0150	1.67961	1741	921	2662	176	91	1515	0.99934
0.85%	0.0200	2.02270	2321	1045	3366	176	80	1526	0.99978
0.82%	0.0025	0.47156	291	326	617	211	205	1439	0.04730
0.82%	0.0050	0.78605	581	521	1102	211	161	1483	0.86115
0.82%	0.0075	1.08038	871	680	1551	211	132	1512	0.98091
0.82%	0.0100	1.33086	1161	807	1968	211	116	1528	0.99440
0.82%	0.0150	1.72105	1741	954	2695	211	104	1540	0.99825
0.82%	0.0200	2.05975	2321	1077	3398	211	95	1549	0.99949
0.80%	0.0025	0.46209	291	319	610	234	236	1421	0.00899
0.80%	0.0050	0.78114	581	522	1103	234	181	1476	0.78147
0.80%	0.0075	1.07974	871	682	1553	234	155	1502	0.96635
0.80%	0.0100	1.33612	1161	816	1977	234	137	1520	0.99021
0.80%	0.0150	1.72292	1741	958	2699	234	116	1541	0.99715
0.80%	0.0200	2.06983	2322	1085	3407	234	103	1554	0.99912

The broad sweep consistently favored narrow rescue regions, with $\epsilon_{\text{res}} = 0.0025$ yielding the highest recovery among the initially evaluated fractions. We therefore examined an even narrower setting $\epsilon_{\text{res}} = 0.002$ together with 0.005 and 0.010. Table A2 shows that within this narrower refinement, $\epsilon_{\text{res}} = 0.002$ achieved the highest recovery at each examined internal allocation while admitting the fewest benign samples. Relative to the $\epsilon_{\text{res}} = 0.0025$ setting, it reduced benign exposure from approximately 290 to 232 samples at the examined allocations, with only a small reduction in recovered false negatives. Therefore, we retained $\epsilon_{\text{res}} = 0.002$ as the most selective rescue region setting for subsequent OOF rescue-data construction and adaptive C-GUARD development.

Table A2. Refined rescue-region benign exposure fraction. δ_{raw} denotes the LightGBM raw score distance width induced by the corresponding benign exposure fraction ϵ_{res}. The selected setting at each internal FPR allocation is underlined.

Internal FPR	ϵ_{res}	δ_{raw}	Region TN	Region FN	Region Size	Extra FP	Recovered FN
0.98%	0.002	0.27203	<u>232</u>	<u>184</u>	<u>416</u>	24	<u>14</u>
0.98%	0.005	0.63426	580	382	962	24	8
0.98%	0.010	1.12025	1159	616	1775	24	10
0.95%	0.002	0.29553	<u>232</u>	<u>210</u>	<u>442</u>	59	<u>45</u>
0.95%	0.005	0.65615	580	412	992	59	34
0.95%	0.010	1.15865	1159	655	1814	59	34
0.90%	0.002	0.33562	<u>232</u>	<u>243</u>	<u>475</u>	117	<u>100</u>
0.90%	0.005	0.70428	580	465	1045	117	73
0.90%	0.010	1.22967	1160	716	1876	117	58

Appendix A.2. Adaptive Conformal Gate Selection and Held-Out Admission Audit

The adaptive conformal gate construction and parameter-selection procedure are described in Algorithm 1 and Section 4.2. To avoid repeating the methodological description, Table A3 reports the validation-selected operating point for each candidate context count $k \in \{1, \ldots, 10\}$ for the LightGBM auxiliary trained on the full EMBER feature representation. For each k, candidate conformal thresholds α were generated at the membership-changing boundaries induced by the distinct cross-conformal malware-class compatibility values among baseline-negative validation samples. Thus, all distinct validation gate memberships induced by the observed compatibility values were evaluated rather than a coarse predefined threshold grid. A complementary descriptive held-out gate-admission audit is reported subsequently in Table A4.

Table A3. Validation-only adaptive conformal gate search summary using the LightGBM auxiliary trained on the full EMBER feature representation. For each candidate number of rescue-score quantile contexts k, the table reports the best feasible operating point obtained after evaluating the corresponding conformal and auxiliary decision thresholds under $B_{\text{gate}} = 234$ gated validation true negatives and $B_{\text{FP}} = 24$ additional false positives. The selected configuration is underlined.

k	α	Gated TN	Gated FN	Total Gate	τ_r	Extra FP	Recovered FN	Final FN
1	0.891916	179	153	332	0.557646	24	45	1341
2	0.903210	144	135	279	0.567504	24	50	1336
3	0.910169	135	132	267	0.587816	24	46	1340
4	0.912903	119	124	243	0.567504	24	52	1334
5	0.921017	116	118	234	0.560314	24	53	1333
6	0.916461	120	122	242	0.569028	24	51	1335
7	0.925764	111	113	224	0.567504	23	54	1332
8	0.915881	116	126	242	0.567504	24	54	1332
9	0.917346	117	126	243	0.559706	24	57	1329
10	0.929495	107	107	214	0.560314	24	51	1335

Among the ten candidate context configurations, $k = 9$ achieved the highest validation false-negative recovery, recovering 57 false negatives while satisfying both operational constraints, and was therefore selected and frozen as the primary C-GUARD configuration.

Table A4 provides a descriptive held-out gate admission audit for the validation selected operating point associated with each candidate context count k. Standard test benign and malware entries correspond, respectively, to baseline true negatives and baseline false negatives admitted to the adaptive gate. The challenge set contains malware only, so its reported count denotes baseline negative challenge malware admitted to the gate. These held-out counts were computed after the corresponding validation configurations had been fixed and were not used to select k, α, or τ_r.

Table A4. Post hoc held-out gate admission audit across C-GUARD candidate context counts using validation selected operating points. Standard test benign and malware counts denote baseline true negatives and baseline false negatives admitted to the adaptive gate, respectively. The challenge set contains malware only. The primary validation selected configuration ($k^* = 9$) is underlined.

k	Standard-Test Benign	Standard-Test Malware	Standard-Test Total	Challenge Malware
1	396	419	815	94
2	334	417	751	83
3	282	377	659	73
4	267	363	630	70
5	267	370	637	69
6	259	354	613	70
7	241	335	576	60
8	277	380	657	68
9	280	368	648	71
10	242	333	575	61

For the primary validation-selected configuration $k^* = 9$, the adaptive gate admitted 280 standard-test benign samples, 368 standard-test malware false negatives,

and 71 challenge-malware false negatives. These counts characterize the held-out gate-admission profile of the frozen primary C-GUARD configuration.

Appendix B. Multi-FPR Selection and Held-Out Evaluation Details

Table A5 reports the validation-selected configurations for the five nominal baseline FPR targets considered in Section 4.4. For each operating point, the baseline threshold was selected on validation, after which k, α, and τ_r were selected independently under the same constraints as the primary experiment. The auxiliary rescue detector remained fixed throughout the analysis. All five selected configurations reached the additional FP limit of 24, while the number of gated validation true negatives remained within the predefined limit of 234. For the threshold-relaxation baseline, the decision threshold was selected on validation to match the final C-GUARD false-positive count. An exact FP match was obtained at all five operating points. The nominal FPR targets refer to the baseline; the corresponding achieved C-GUARD validation FPR is reported in a separate column.

Table A5. Validation-selected configurations and the FP-matched threshold-relaxation baseline across low-FPR operating points. FP and FN denote false positives and false negatives, respectively; Gate TN/FN reports the validation samples admitted to the adaptive gate.

Nominal FPR	τ_b	Baseline FP/FN	k	α	τ_r	Gate TN/FN	C-GUARD FP/FN	C-GUARD FPR	Matched τ	Matched FN
0.10%	3.369822	117/7019	8	0.665758	0.500134	105/2356	141/5991	0.1205%	3.083151	6190
0.25%	2.090996	292/3995	4	0.760985	0.531250	115/962	316/3635	0.2701%	1.960293	3768
0.50%	0.987833	585/2366	4	0.819947	0.581966	171/430	609/2230	0.5205%	0.938160	2310
0.75%	0.365875	877/1752	1	0.911090	0.515779	126/161	901/1674	0.7701%	0.322583	1710
1.00%	−0.126163	1170/1386	9	0.917346	0.559706	117/126	1194/1329	1.0205%	−0.158669	1368

Table A6 reports the standard test and challenge set results obtained using the thresholds and adaptive parameters selected on the validation split. The standard test FPR may differ from the nominal validation targets. The threshold-relaxation baseline, likewise, remains fixed after validation and is not re-matched using held-out labels.

Table A6. Multi-FPR evaluation results. Standard test results are reported as FP/FN/FPR. The threshold-relaxation baseline is FP-matched on validation and applied unchanged to the held-out data. Challenge set entries report false negatives for the baseline (B), C-GUARD (C), and matched baseline (M).

Nominal	Standard Test: FP/FN/FPR			Challenge FN
FPR	Baseline	C-GUARD	Validation-Matched Threshold	B / C / M
0.10%	375/20,710/0.1389%	491/17,603/0.1819%	474/18,635/0.1756%	2763/2591/2677
0.25%	867/12,666/0.3211%	987/11,563/0.3656%	956/12,018/0.3541%	2380/2265/2337
0.50%	1717/8076/0.6359%	1816/7627/0.6726%	1772/7915/0.6563%	1939/1898/1910
0.75%	2471/6179/0.9152%	2556/5961/0.9467%	2536/6055/0.9393%	1675/1649/1657
1.00%	3188/5059/1.1807%	3270/4881/1.2111%	3246/4992/1.2022%	1408/1391/1395

C-GUARD produces fewer false negatives than the validation-matched threshold-relaxation baseline at every operating point on both the standard test and evasive challenge sets. FP matching is performed only on validation; these held-out results reflect the performance of the frozen validation selected configurations rather than a test-set FP-matched comparison.

Appendix C. Validation Stability of the Adaptive Gate

As a complementary analysis to the held-out statistical reliability results reported in Section 4.3, we assess the stability of the validation selected adaptive gate using 5000 paired

stratified bootstrap replicates. This analysis evaluates the stability of the selected operating point without retraining the auxiliary detector or retuning the frozen operating parameters. For $k = 9$, the observed 57 recovered false negatives correspond to a bootstrap mean of 56.9 and a 95% interval of [43, 72]. The $k = 9$ configuration is selected in 61.28% of bootstrap resamples, compared with 15.12%, 13.96%, and 9.64% for $k = 7$, $k = 8$, and $k = 5$, respectively. Relative to the fixed-margin gate, $k = 9$ produces a mean gain of 14.0 recovered false negatives, with a 95% bootstrap interval of [4, 25]. These results characterize the stability of the validation selected operating point.

References

1. Redhu, A.; Choudhary, P.; Srinivasan, K.; Das, T.K. Deep learning-powered malware detection in cyberspace: A contemporary review. *Front. Phys.* **2024**, *12*, 1349463. [CrossRef]
2. Al-Essa, M.; Franchini, F.; Galantucci, S.; Imran, M.; Pirlo, G. Improving robustness and explainability of PE malware classifiers using GAN-Generated Synthetic Adversarial examples. *Cybersecurity* **2026**, *9*, 169. [CrossRef]
3. Joyce, R.J.; Miller, G.; Roth, P.; Zak, R.; Zaresky-Williams, E.; Anderson, H.; Raff, E.; Holt, J. EMBER2024—A Benchmark Dataset for Holistic Evaluation of Malware Classifiers. In Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, Toronto ON, Canada, 3–7 August 2025; pp. 5516–5526.
4. To, T.N.; Kim, D.L.; Hien, D.T.T.; Khoa, N.H.; Hoang, H.D.; Duy, P.T.; Pham, V.H. On the effectiveness of adversarial samples against ensemble learning-based windows PE malware detectors. *Int. J. Inf. Secur.* **2025**, *24*, 60. [CrossRef]
5. Ling, X.; Wu, Z.; Wang, B.; Deng, W.; Wu, J.; Ji, S.; Luo, T.; Wu, Y. A wolf in sheep's clothing: Practical black-box adversarial attacks for evading learning-based windows malware detection in the wild. In Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24), Philadelphia, PA, USA, 14–16 August 2024; pp. 7393–7410.
6. Saqib, M.; Mahdavifar, S.; Fung, B.C.; Charland, P. A comprehensive analysis of explainable AI for malware hunting. *ACM Comput. Surv.* **2024**, *56*, 1–40. [CrossRef]
7. Šimić, I.; Veas, E.; Sabol, V. A comprehensive analysis of perturbation methods in explainable AI feature attribution validation for neural time series classifiers. *Sci. Rep.* **2025**, *15*, 26607. [CrossRef] [PubMed]
8. Lundberg, S.M.; Lee, S.I. A unified approach to interpreting model predictions. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 4765–4774.
9. Ribeiro, M.T.; Singh, S.; Guestrin, C. " Why should i trust you?" Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 1135–1144.
10. Imran, M.; Appice, A.; Malerba, D. Evaluating realistic adversarial attacks against machine learning models for Windows PE Malware Detection. *Future Internet* **2024**, *16*, 168. [CrossRef]
11. Kamdan; Pratama, Y.; Munzi, R.S.; Mustafa, A.B.; Kharisma, I.L. Static Malware Detection and Classification Using Machine Learning: A Random Forest Approach. *Eng. Proc.* **2025**, *107*, 76. [CrossRef]
12. Van Ouytsel, C.H.B.; Dam, K.H.T.; Legay, A. Analysis of machine learning approaches to packing detection. *Comput. Secur.* **2023**, *136*, 103536. [CrossRef]
13. Abdrakhmanov, B. Hybrid AI-Based Static Analysis for Malware Detection: A Feature Engineering and Model Optimization Approach. In *Proceedings of the 2025 IEEE 5th International Conference on Smart Information Systems and Technologies (SIST)*; IEEE: New York, NY, USA, 2025; pp. 1–6.
14. Saqib, M.; Fung, B.C.; Ding, S.H.; Charland, P. MalGPT: A Generative Explainable Model for Malware Binaries. In *Proceedings of the Joint European Conference on Machine Learning and Knowledge Discovery in Databases*; Springer: Cham, Switzerland, 2025; pp. 130–148.
15. Alani, M.M.; Mashatan, A.; Miri, A. XMal: A lightweight memory-based explainable obfuscated-malware detector. *Comput. Secur.* **2023**, *133*, 103409. [CrossRef]
16. Mohammadian, H.; Higgins, G.; Ansong, S.; Razavi-Far, R.; Ghorbani, A.A. Explainable malware detection through integrated graph reduction and learning techniques. *Big Data Res.* **2025**, *41*, 100555. [CrossRef]
17. Vo, H.V.; Du, H.P.; Nguyen, H.N. MDOB: Enhancing resilient and explainable AI-powered malware detection using feature set optimization and Mutual Deep+ Boosting Ensemble Inference. *J. Inf. Secur. Appl.* **2025**, *93*, 104175. [CrossRef]
18. AL-Essa, M.; Andresini, G.; Appice, A.; Malerba, D. XAI to Explore Robustness of Features in Adversarial Training for Cybersecurity. In *Proceedings of the Foundations of Intelligent Systems*; Springer International Publishing: Cham, Switzerland, 2022; pp. 117–126.
19. Sundararajan, M.; Taly, A.; Yan, Q. Axiomatic attribution for deep networks. In Proceedings of the International Conference on Machine Learning, PMLR, Sydney, Australia, 6–11 August 2017; pp. 3319–3328.

20. Bach, S.; Binder, A.; Montavon, G.; Klauschen, F.; Müller, K.R.; Samek, W. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PLoS ONE* **2015**, *10*, e0130140. [CrossRef] [PubMed]
21. Rosenberg, I.; Meir, S.; Berrebi, J.; Gordon, I.; Sicard, G.; David, E.O. Generating end-to-end adversarial examples for malware classifiers using explainability. In *Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN)*; IEEE: New York, NY, USA, 2020; pp. 1–10.
22. Al-Essa, M.; Ali, W.A.; Alsharo, M.; Atoum, M.S.; Imran, M. COBRA: Counterfactual Oversampling Framework for Imbalanced Structured Data Classification. *Expert Syst. Appl.* **2025**, *304*, 130764. [CrossRef]
23. Aryal, K.; Gupta, M.; Abdelsalam, M.; Saleh, M. Explainability guided adversarial evasion attacks on malware detectors. In *Proceedings of the 2024 33rd International Conference on Computer Communications and Networks (ICCCN)*; IEEE: New York, NY, USA, 2024; pp. 1–9.
24. Demetrio, L.; Coull, S.E.; Biggio, B.; Lagorio, G.; Armando, A.; Roli, F. Adversarial exemples: A survey and experimental evaluation of practical attacks on machine learning for windows malware detection. *ACM Trans. Priv. Secur. (TOPS)* **2021**, *24*, 1–31.
25. Qiao, Y.; Zhang, W.; Tian, Z.; Yang, L.T.; Liu, Y.; Alazab, M. Adversarial malware sample generation method based on the prototype of deep learning detector. *Comput. Secur.* **2022**, *119*, 102762. [CrossRef]
26. Aryal, K.; Gupta, M.; Abdelsalam, M.; Saleh, M. Intra-section code cave injection for adversarial evasion attacks on windows pe malware file. *Comput. Secur.* **2025**, *159*, 104690. [CrossRef]
27. Zhan, D.; Duan, Y.; Hu, Y.; Li, W.; Guo, S.; Pan, Z. MalPatch: Evading DNN-based malware detection with adversarial patches. *IEEE Trans. Inf. Forensics Secur.* **2023**, *19*, 1183–1198. [CrossRef]
28. Mekdad, Y.; Naseem, F.; Aris, A.; Oz, H.; Acar, A.; Babun, L.; Uluagac, S.; Tuncay, G.S.; Ghani, N. On the robustness of image-based malware detection against adversarial attacks. In *Network Security Empowered by Artificial Intelligence*; Springer: Cham, Switzerland, 2024; pp. 355–375.
29. Salman, M.; Zhao, B.Z.H.; Asghar, H.J.; Ikram, M.; Kaushik, S.; Kaafar, M.A. On the robustness of malware detectors to adversarial samples. In *Proceedings of the European Symposium on Research in Computer Security*; Springer: Cham, Switzerland, 2024; pp. 149–170.
30. Kozák, M.; Jureček, M.; Stamp, M.; Troia, F.D. Creating valid adversarial examples of malware. *J. Comput. Virol. Hacking Tech.* **2024**, *20*, 607–621. [CrossRef]
31. Kozák, M.; Jureček, M. Combining generators of adversarial malware examples to increase evasion rate. *arXiv* **2023**, arXiv:2304.07360.
32. Song, W.; Li, X.; Afroz, S.; Garg, D.; Kuznetsov, D.; Yin, H. Mab-malware: A reinforcement learning framework for blackbox generation of adversarial malware. In Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May–3 June 2022; pp. 990–1003.
33. Hu, W.; Tan, Y. Generating adversarial malware examples for black-box attacks based on GAN. In *Proceedings of the International Conference on Data Mining and Big Data*; Springer: Singapore, 2022; pp. 409–423.
34. Jia, L.; Yang, Y.; Li, J.; Ding, H.; Li, J.; Yuan, T.; Liu, L.; Jiang, Z. MTMG: A framework for generating adversarial examples targeting multiple learning-based malware detection systems. In *Proceedings of the Pacific Rim International Conference on Artificial Intelligence*; Springer: Singapore, 2023; pp. 249–261.
35. Akiba, T.; Sano, S.; Yanase, T.; Ohta, T.; Koyama, M. Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 2623–2631.
36. Watanabe, S. Tree-structured parzen estimator: Understanding its algorithm components and their roles for better empirical performance. *arXiv* **2023**, arXiv:2304.11127.
37. Bestgen, Y. Please, don't forget the difference and the confidence interval when seeking for the state-of-the-art status. In Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, 20–25 June 2022; pp. 5956–5962.
38. Rainio, O.; Teuho, J.; Klén, R. Evaluation metrics and statistical tests for machine learning. *Sci. Rep.* **2024**, *14*, 6086. [CrossRef] [PubMed]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.